

Produtor-Consumidor

Produtor-Consumidor

- O padrão produtor-consumidor é uma solução comum para o problema de coordenação entre threads que produzem dados e threads que consomem esses dados.
- Em Java, isso geralmente é implementado usando a sincronização inerente da linguagem, com métodos `synchronized`, `wait()` e `notify()` ou `notifyAll()`.

Produtor-Consumidor

```
3 public class Buffer {
4     private int conteudo;
5     private boolean disponivel = false;
6
7     public synchronized void set(int valor) {
8         while (disponivel) {
9             try {
10                 wait(); // Espera até o buffer estar vazio
11             } catch (InterruptedException e) { }
12         }
13         conteudo = valor;
14         disponivel = true;
15         notifyAll(); // Notifica que o buffer agora está cheio
16     }
17
18     public synchronized int get() {
19         while (!disponivel) {
20             try {
21                 wait(); // Espera até o buffer estar cheio
22             } catch (InterruptedException e) { }
23         }
24         disponivel = false;
25         notifyAll(); // Notifica que o buffer agora está vazio
26         return conteudo;
27     }
28 }
```

Produtor-Consumidor

```
3 public class Consumidor implements Runnable {
4     private Buffer buffer;
5
6     public Consumidor(Buffer buffer) {
7         this.buffer = buffer;
8     }
9
10    public void run() {
11        for (int i = 0; i < 10; i++) {
12            int valor = buffer.get();
13            System.out.println("Consumido: " + valor);
14        }
15    }
16 }
```

Produtor-Consumidor

```
3 public class Produtor implements Runnable {
4     private Buffer buffer;
5
6     public Produtor(Buffer buffer) {
7         this.buffer = buffer;
8     }
9
10    public void run() {
11        for (int i = 0; i < 10; i++) {
12            buffer.set(i);
13            System.out.println("Produzido: " + i);
14        }
15    }
16 }
```

Produtor-Consumidor

```
3 public class Principal {  
4     public static void main(String[] args) {  
5         Buffer bufferCompartilhado = new Buffer();  
6         Thread produtorThread = new Thread(new Produtor(bufferCompartilhado));  
7         Thread consumidorThread = new Thread(new Consumidor(bufferCompartilhado));  
8  
9         produtorThread.start();  
10        consumidorThread.start();  
11    }  
12 }
```